AI
ENGINEERING
AI-Powered products

MÓDULO 3: Data-driven AI

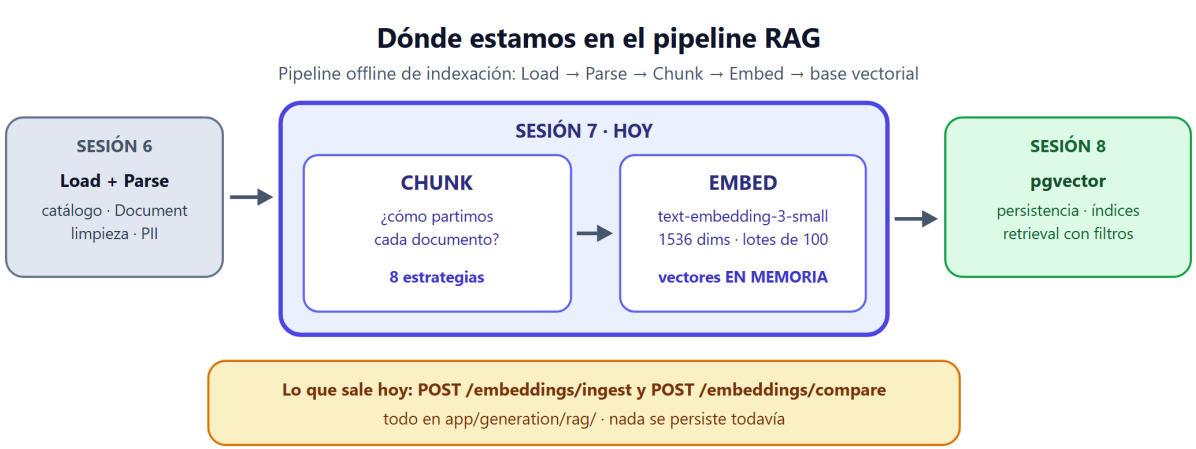
Sesión 7: Embeddings y representación vectorial

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

LIDR

Hoy: Chunk + Embed

Del corpus limpio de la Sesión 6 a los vectores · pgvector llega en la Sesión 8



Al salir de hoy sabréis…

- Qué es un chunk, un token y un vector.
- Elegir modelo de embeddings con criterio.
- Comparar 8 estrategias de chunking con datos.
- Defender una elección ante coste y calidad.

El corpus

17 presupuestos · 60 componentes. Dos con trampa: **MONO-001**, una descripción de 1391 tokens, y un MVP con descripciones de una sola línea.

Cómo trabajamos

Cada estrategia: la idea → el código → la demo.

Teaching Assistant: George Kutsia · dudas en el canal asíncrono.

L1DR

Tres arquitecturas, una base común

Por qué reorganizamos el proyecto en esta sesión · de carpetas planas a arquitectura por capas

ESTRUCTURA DEL PROYECTO

```
app/
├── foundation/        plomería sin opinión
├── domain/            schemas + el "conductor"
├── generation/        las 3 arquitecturas que COMPONEN
│   ├── cag/           CAG   caches (ya construido)
│   ├── rag/           RAG   ● LO DE HOY
│   │   ├── chunking/ · embedding/ · analysis/
│   │   ├── store/         ← slot reservado S8
│   │   └── retriever.py   ← slot reservado S8
│   ├── agentic/       AGENTIC   boss + critic (futuro)
│   └── conversation/
├── ingestion/         pipeline batch que alimenta RAG
└── api/               transporte (routers finos)
```

REGLA: Los hermanos de generation/ no se importan entre sí · solo se encuentran en el conductor

Qué cambia exactamente

El estimador apila tres arquitecturas de IA de forma aditiva a lo largo del máster — CAG, RAG (hoy) y agentic (futuro).

A partir de hoy esa historia se ve en la estructura.

Por qué ahora

Las carpetas planas escondían el patrón: caches partidas, RAG aislado, lo agéntico mezclado en services.

Antes de añadir más piezas, ponemos cada cosa en su capa.

Lo de hoy en este mapa

Todo lo que tocamos en la S7 es exactamente el contenido de **`app/generation/rag/`**

store/ y retriever.py se rellenan en la S8.

Frase para defender ante un stakeholder

"No reescribimos lógica: movimos las piezas a su capa. La arquitectura ahora se lee de un vistazo y cada cosa nueva sabe dónde va."

L1DR

El ejercicio previo: vuestro sanity check

text-embedding-3-small · coseno calculado a mano · tres parejas

Pareja	Texto 1	Texto 2	Coseno	Esperado
A · cercanas	OAuth 2.0 authentication backend with JWT tokens for fintech mobile app	Authorization service using JSON Web Tokens for a banking application	**0,596**	> 0,6
B · sin relación	(la misma)	Database migration from MySQL to PostgreSQL with zero downtime	**0,192**	< 0,4
C · genéricas	Backend services	API development	**0,541**	—

A: 0,596 < 0,6

Mismo concepto, casi sin palabras en común. Los umbrales dependen del modelo y de la redacción.

B: 0,19

Temas sin relación, bien separados. Entre 0,2 y 0,5 es normal para textos que no se parecen.

C: 0,54, el aviso

El texto corto y genérico no discrimina. Por eso cada chunk lleva una cabecera de contexto.

L1DR

Cinco patrones de error reales

Ordenados por impacto en producción

1. **Sin validación Pydantic**
   Síntoma: `KeyError` opaco en producción cuando falta un campo
   Resuelto en · app/generation/rag/schemas.py

2. **Gestión naïve de tokens**
   Síntoma: chunk excede contexto del modelo · `422` de la API de embeddings
   Resuelto en · app/generation/rag/chunking/base.py :: count_tokens

3. **Sin batching**
   Síntoma: un request por chunk · latencia que se dispara · throughput inviable
   Resuelto en · embedder.py :: BATCH_SIZE = 100 + embed_many

4. **Sin backoff exponencial**
   Síntoma: caída ante el primer `RateLimitError` · sin resiliencia
   Resuelto en · embedder.py :: _RETRY_BACKOFF_SECONDS = (1, 2, 4)

5. **Dimensionalidad inconsistente**
   Síntoma: mezclar vectores de 1536 y 768 en el mismo índice rompe la similitud
   Resuelto en · schemas.py :: EmbeddedChunk con campo embedding tipado

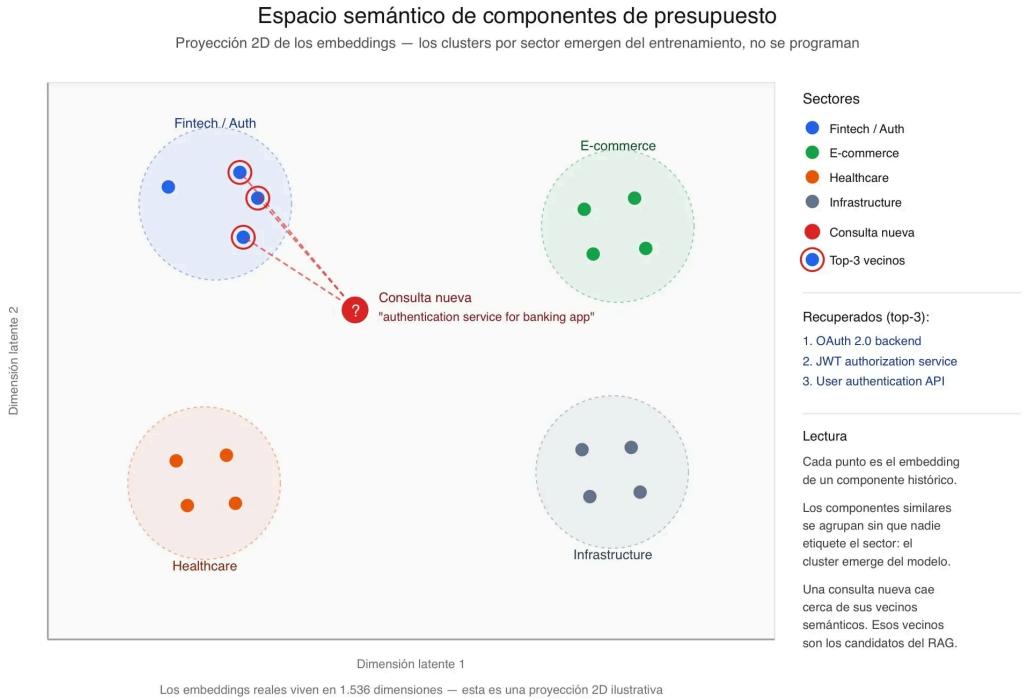
Pregunta gancho del bloque

"Vuestro chunker no truncó
esa descripción de 1.391 tokens.
¿Está bien o está mal?"

Respuesta esperada: depende.
Hoy no rompe. Mañana sí.

Chunk, token y vector

Las tres palabras que usamos todo el rato



Chunk

El trozo de documento que se indexa y se recupera como una unidad. Es lo que el RAG le da al LLM.

Token

La unidad en la que el modelo cuenta el texto: unos ¾ de palabra. Una página llena ≈ 700–1000 tokens.

Vector (embedding)

1536 números por chunk. Textos parecidos → vectores cercanos, aunque no compartan palabras.

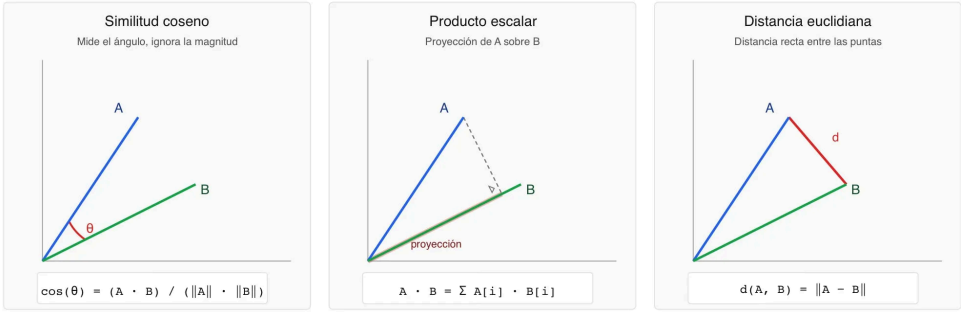
Si un chunk está mal cortado, su vector lo representa mal… y ningún modelo lo arregla después.

Medir la similitud y sus límites

Con vectores normalizados, las tres métricas ordenan igual

Tres formas de medir la similitud entre dos vectores

Los mismos dos vectores A y B vistos bajo cada métrica



Para vectores normalizados (lo habitual en modelos modernos), las tres métricas ordenan los resultados de la misma forma

Coseno

Mide el ángulo, no la longitud. `text-embedding-3-small` devuelve vectores de norma 1, así que producto escalar = coseno. La métrica la marca el modelo, no es una decisión de arquitectura.

Lo que un embedding NO resuelve

- **Números y fechas** → filtros sobre el metadata (S8).
- **Palabras raras exactas** → búsqueda híbrida con BM25 (S10).
- **Umbrales** → se calibran con tus datos.

Elegir modelo de embeddings

Precios y scores cambian cada pocos meses: verificar antes de decidir

Modelo	Dims	$ / M tokens	Notas
text-embedding-3-small	1536	0,02	el caballo de batalla · Matryoshka
text-embedding-3-large	3072	0,13	mejor calidad, 6,5× más caro
Cohere embed-v3	—	0,10	fuerte en multilingüe
Voyage voyage-3-large	—	≈ Cohere	optimizado para retrieval
BAAI/bge-m3	1024	0 (GPU)	open source, multilingüe
all-MiniLM-L6-v2	384	0 (CPU)	inglés, para prototipos

Cinco criterios

- **Dimensiones**: ¿cabe en RAM?
- **Idioma**: briefs en español → multilingüe.
- **Dominio**: se valida sobre tus datos.
- **Coste**: API o self-hosted.
- **Privacidad**: banca, sanidad → self-hosted.

Nuestra decisión

`text-embedding-3-small`, 1536 dims: calidad sobrada, coste despreciable.

L1DR

Matryoshka: la palanca de dimensiones

Mismo modelo · misma señal · la mitad de storage

Modelo	Dimensiones	Latencia media	Cosine(c0, c1)	Δ Calidad
`text-embedding-3-small` @ 1536d nativo	1536	879.8 ms	0.9119	baseline
`text-embedding-3-small` @ 768d (Matryoshka)	768	699.9 ms	0.9117	–0.0002

½ storage

Mitad de dimensiones = mitad de bytes por vector en la BBDD

Para 1M de chunks: GB que entran en RAM o que no entran

Sin pérdida real

Cosine entre dos componentes similares: idéntica hasta el cuarto decimal

0.9119 vs 0.9117 — ruido

Un parámetro

Truncar dimensiones no es un modelo nuevo. Es el argumento **`dimensions=768`**

Palanca futura para S8

LOCKED PARA EL PROYECTO

text-embedding-3-small @ 1536d · API ya configurada · coste despreciable · Matryoshka como palanca futura

¿Por qué no se comparan vectores de distinta dimensión?

1536 contra 768 no da un número malo: da un número sin sentido

La matemática

El producto escalar `Σ A[i]·B[i]` empareja la coordenada *i* de uno con la del otro. Con 1536 frente a 768, **las 768 últimas no tienen pareja**: son espacios distintos.

Si tienes suerte: el error salta

```
>>> np.dot(v1536, v768)
ValueError: shapes (1536,) and (768,) not aligned

-- pgvector (Sesión 8)
ERROR: different vector dimensions 1536 and 768
```

Si no la tienes: nuestro coseno en Python puro

```
# app/generation/rag/analysis/similarity.py
dot = sum(x * y for x, y in zip(a, b))
#                           ^^^ zip se para en la
#                     lista más corta: sin error
```

El bug silencioso, medido

Un vector de 1536 contra **sus propias** 768 primeras dimensiones:

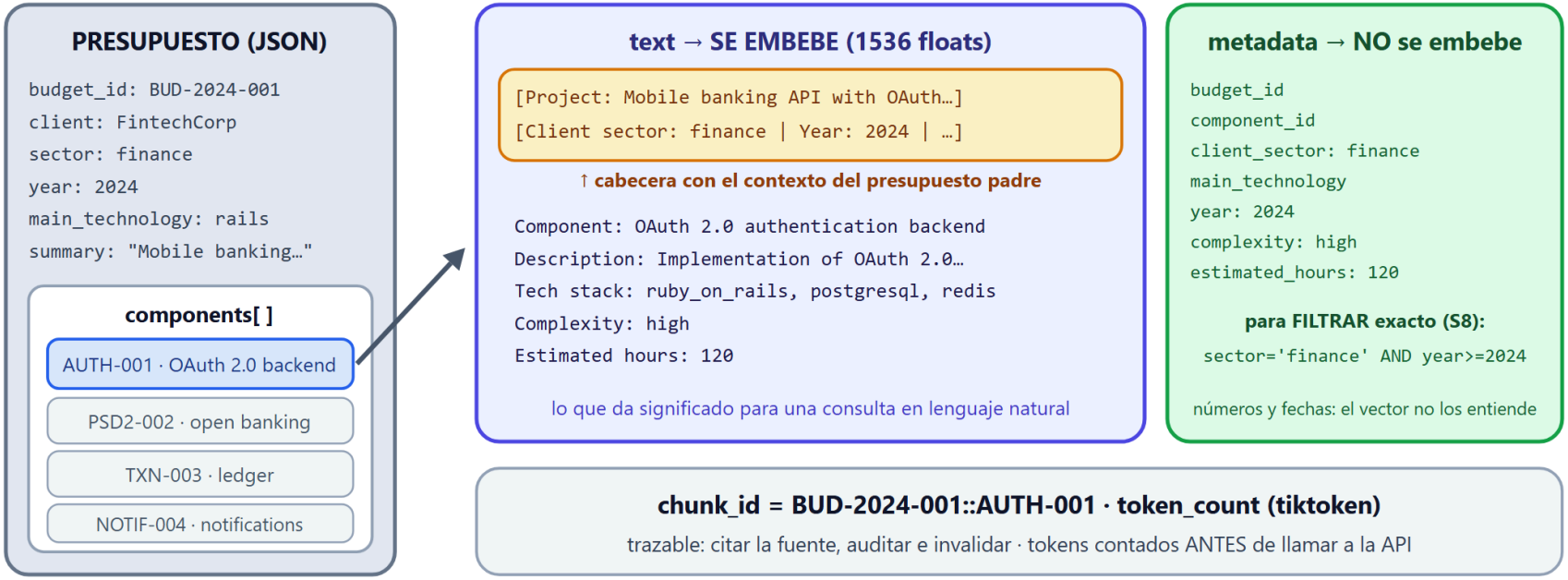
0,689 en vez de **1,0**

Cómo se evita: un modelo y una dimensión por índice · `assert len(a) == len(b)` · dos modelos distintos con 1536 dims tampoco son comparables.

El chunk de nuestro proyecto

app/generation/rag/chunking/structural.py · la estrategia de referencia

Anatomía de un chunk: qué se embebe y qué viaja al lado



1 componente = 1 chunk

Sector, año y horas van en los dos: en el texto dan contexto; en el metadata, filtro exacto

El componente es lo que estimas; el chunk es lo que buscas. Con structural, son lo mismo.

L1DR

Las cuatro familias de chunking

Ocho estrategias agrupadas por cómo deciden dónde partir el texto

1 Mecánicas

Cortan por reglas · ignoran el contenido

Bloque 4 — 30 min

- FixedSize — baseline degenerado
- Recursive — default razonable
- SentenceWindow — oración indexada · ventana devuelta

Coste: $0

2 Estructurales

Explotan el formato del documento

Bloque 4 (parcial) + Bloque 6

- JSONStructural — un componente = un chunk
- Hierarchical — parent-child · dos niveles

Coste: $0

3 Semánticas

Cortan donde cambia el significado

Bloque 5 — primera mitad

- Semantic — breakpoint por caída de similitud
- Propositional — LLM descompone en proposiciones

Coste real · medir ROI

4 Contextuales

Enriquecen el chunk antes de embedear

Bloque 5 + Bloque 6

- ContextualRetrieval — Claude + prompt caching
- LateChunking — demo conceptual

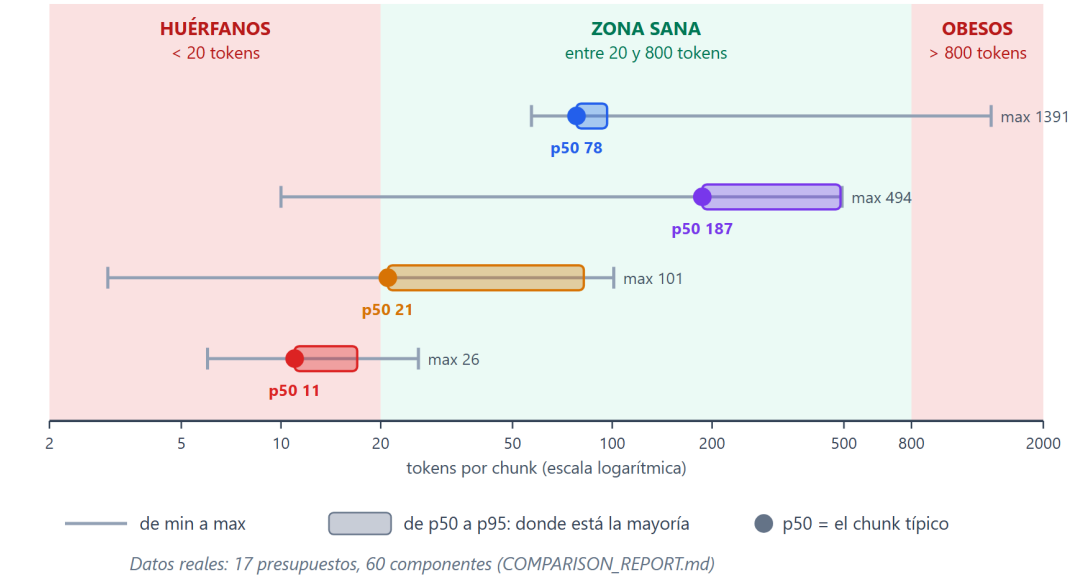
Coste premium

Todas implementan la misma interfaz Chunker · intercambiables en el comparador

Cómo leer el Chunking Lab

El mismo corpus, cuatro formas de partirlo

Tamaño de los chunks en tokens: dónde cae cada estrategia



Datos reales: 17 presupuestos, 60 componentes (COMPARISON_REPORT.md)

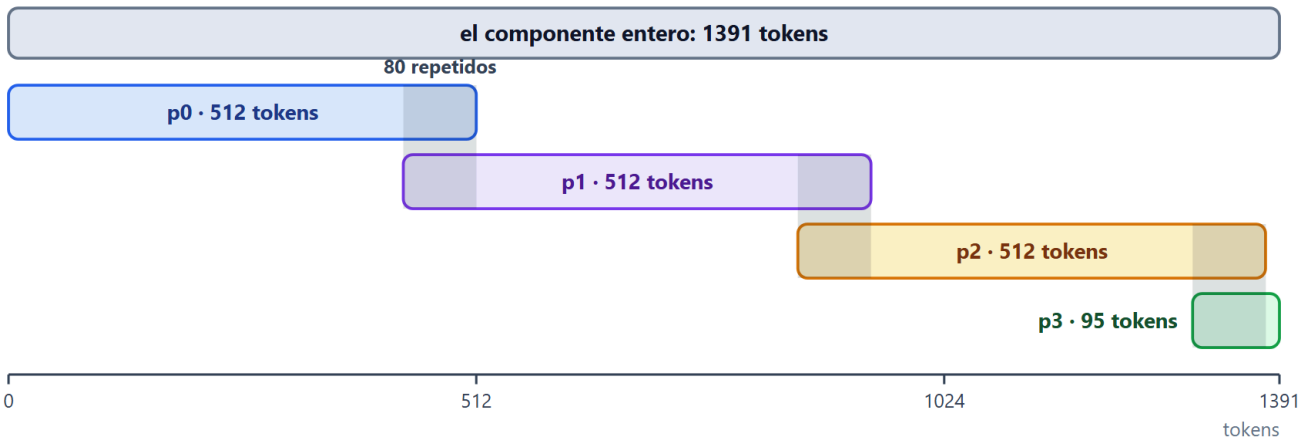
Las columnas

- **p50**: el chunk típico; la mitad mide menos.
- **p95**: el 95 % mide menos. Lejos de p50 = tamaños irregulares.
- **Huérfano** (< 20 tokens): **malo**, sin contexto.
- **Obeso** (> 800 tokens): **malo**, mezcla temas.
- **Coste y segundos**: lo que cuesta la ingesta.

Lo que buscamos: cero huérfanos, cero obesos y un p50 con sentido para el documento.

1/8 · Fixed-size

Familia mecánica



Dónde cae el corte (texto real)

p3 empieza en «it knowledge is the single largest project risk…»: la palabra anterior quedó partida.

Los otros 59 componentes miden menos de 512 tokens: ahí no corta nada (1 componente = 1 chunk)

La idea

Cuenta 512 tokens y corta, sin mirar frases ni componentes. Repite 80 entre trozos.

Úsala para

Línea base para comparar. Texto sin ninguna estructura.

Cuidado con

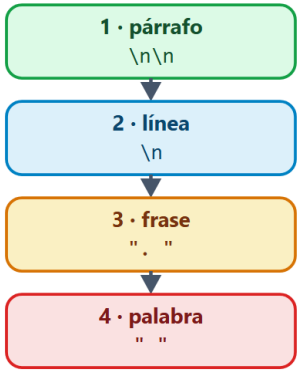
Parte palabras y frases. Aquí solo actúa en MONO-001.

2/8 · Recursive

Familia mecánica

Recursive: corta por el separador más grande que pueda

Solo baja al siguiente separador si el trozo sigue pasando de 512 tokens



BUD-2024-001 · 278 tokens → cabe entero

r0 · un solo chunk

AUTH-001 | PSD2-002 | TXN-003 | NOTIF-004

4 componentes distintos, un solo vector

BUD-2024-016 · 1472 tokens → 7 chunks

- r0 · 41 tok · cabecera
- **r1 · 10 tok · solo el nombre**
- r2 · 457 tok · descripción 1/3
- r3 · 492 tok · descripción 2/3
- r4 · 494 tok · descripción 3/3
- r5 · 27 tok · stack y horas
- r6 · 74 tok · MONO-002 entero

r1 es un huérfano (10 tokens)

En nuestro corpus: 23 chunks para 17 presupuestos · casi todos caben enteros en uno

La idea

Corta por párrafos; si sigue siendo grande, por líneas, frases y, al final, palabras.

Úsala para

El punto de partida cuando no hay estructura: 400–512 tokens.

Cuidado con

Si el documento cabe entero, lo deja en un chunk y mezcla temas.

L1DR

Fixed-size vs Recursive

Mismo presupuesto serializado · dos formas radicalmente distintas de partirlo

FixedSize · CHUNK_SIZE_TOKENS = 512 · OVERLAP = 80

BUD-2024-016::MONO-001::p0 · 512 tokens

```
[Project: Migration of legacy banking platform...] Component:
Core banking monolith decomposi tion into 18 microservices
Description: Strangler-fig migration of a 12-year-old Java mono
lith handling 2.4M daily transactions. The migration plan spa
ns 18 months and inclu
```

BUD-2024-016::MONO-001::p1 · 512 tokens

```
des the following dom ains: account management, transactional
core, identity services, fraud detection, audit, regulatory r
eporting, KYC compliance, AML monitoring, customer profiling,
notification dispatch, scheduled batches, and external integ
rations with PSD2-com
```

BUD-2024-016::MONO-001::p2 · 367 tokens

```
pliant providers. Each domain will be implemented as an in
dependent service with its own database and event bus...
[...]
```

63 chunks · p50: 79 tok · max: 512 tok · obesos: 0

"decomposi tion" · "dom ains" · palabras rotas a mitad

Recursive · separadores [\n\n, \n, ". ", " ", ""]

BUD-2024-016::r0 · 488 tokens

```
[Project: Migration of legacy banking platform to cloud-native
microservices architecture]

Component: Core banking monolith decomposition into
18 microservices.
Description: Strangler-fig migration of a 12-year-old Java...
```

BUD-2024-016::r1 · 494 tokens

```
...handling 2.4M daily transactions.

The migration plan spans 18 months and includes the
following domains: account management, transactional core,
identity services, fraud detection, audit, regulatory
reporting, KYC compliance...
```

BUD-2024-016::r2 · 187 tokens

```
...AML monitoring, notification dispatch, scheduled batches,
and external integrations with PSD2-compliant providers.
```

23 chunks · p50: 187 tok · max: 494 tok · obesos: 0

Oraciones intactas · respeta saltos naturales del lenguaje

El default razonable: Recursive con 400–512 tokens y 10–20% de overlap

Solo se complejiza con evidencia medida de que no es suficiente · MVP con un día de plazo: este es el chunker

Sentence-window: se busca por frase, se devuelve la ventana

Ejemplo real: MONO-001 partido en frases (WINDOW_SIZE = 2)

s0 This component covers the full extraction of the general ledger...

s1 The work begins with a complete reverse-engineering exercise...

s2 Each stored procedure is catalogued, its inputs and outputs...

s3 Once the golden master is in place, the new service is built...

s4 Then the write path is migrated account-product by account-product...

s5 Every write is dual-written to both systems during a bake-in...

s6 The component also includes the design of the event schema...

ventana ±2
(metadata.window_text)
→ es lo que recibe el LLM

s3 → lo único que se embebe y se busca

El riesgo

BUD-2024-017 · MVP-001

«Email and password login.»

5 tokens

Sale #2 (0,441) para «OAuth authentication for fintech mobile app»

sin saber que es un MVP

En nuestro corpus: 78 chunks · p50 de 21 tokens · 34 huérfanos

La idea

Cada frase es un chunk. Se busca por la frase y al LLM se le da con sus vecinas.

Úsala para

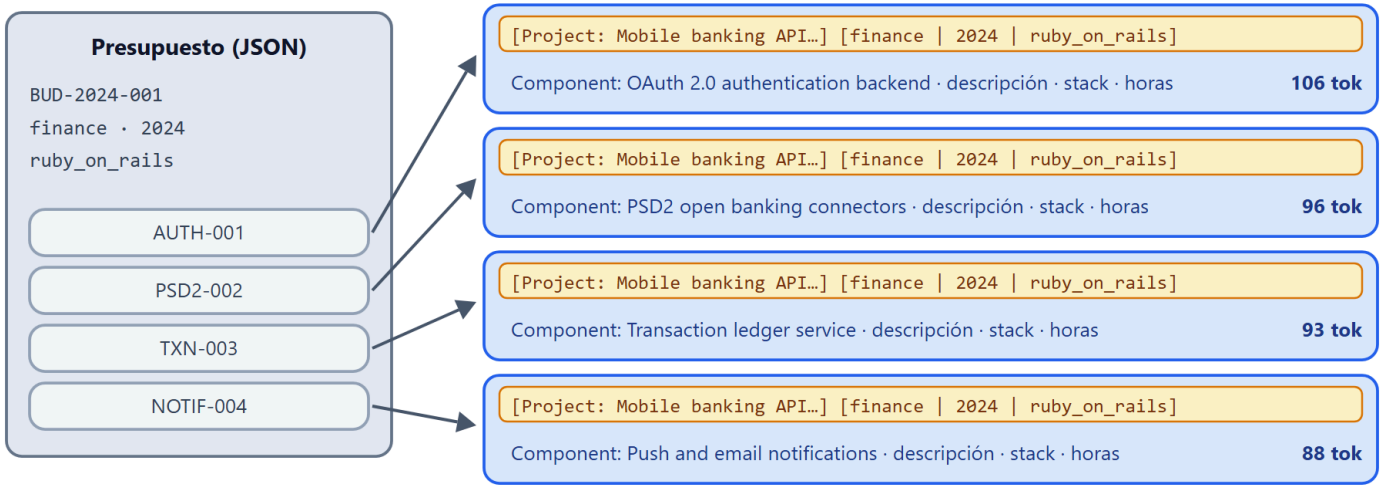
Prosa larga donde una frase concreta responde.

Cuidado con

Frases cortas sin contexto: huérfanos con score alto que no sirven.

Structural: un componente = un chunk, con la cabecera del padre

Ejemplo real: BUD-2024-001 → 4 chunks



En nuestro corpus: 60 chunks · p50 de 78 tokens · 1 obeso (MONO-001: no lo parte, lo deja ver) · 0 $

La idea

Respeta la forma del documento: un componente del JSON es un chunk, con su cabecera.

Úsala para

Datos con estructura: JSON, Markdown, HTML. La palanca de mayor retorno.

Cuidado con

No parte lo demasiado grande: MONO-001 entra entero.

L1DR

Sentence-window vs Structural

Misma consulta · resultados que parecen iguales pero esconden la trampa del score alto

CONSULTA: **"OAuth authentication for fintech mobile app"**

SentenceWindow · indexa oraciones sueltas

#1 [0.4530] BUD-2024-001::PSD2-002::s0
"Implements PSD2 strong customer authentication..."
relevante

#2 [0.4410] BUD-2024-017::MVP-001::s0
"Email and password login."
⚠ del MVP de e-commerce — score alto, contexto perdido

#3 [0.4110] BUD-2024-006::VENDOR-001::s0
"Third-party vendor onboarding with SSO..."
parcialmente relevante

JSONStructural · un componente = un chunk

#1 [0.5684] BUD-2024-001::AUTH-001
[Project: Mobile banking API...] OAuth 2.0 backend...
✓ el componente OAuth correcto con contexto

#2 [0.5310] BUD-2024-001::NOTIF-004
[Project: Mobile banking API...] Notification svc...
mismo proyecto · relacionado

#3 [0.5220] BUD-2024-001::PSD2-002
[Project: Mobile banking API...] PSD2 SCA module...
✓ relacionado, mismo dominio

Lo que el sentence-window oculta

Indexa oraciones diminutas y descontextualizadas · "Email and password login." parece auth, no lo es
La oración recuperada NO es lo que pasarías al LLM-generador · eso vive en **`metadata.window_text`**
Score alto ≠ chunk relevante. La unidad indexada debe ser distinta de la devuelta cuando la indexada pierde sentido aislada.

Semantic: corta donde más cambia el significado

Ejemplo real: BUD-2024-001 · distancia entre trozos consecutivos = 1 − coseno

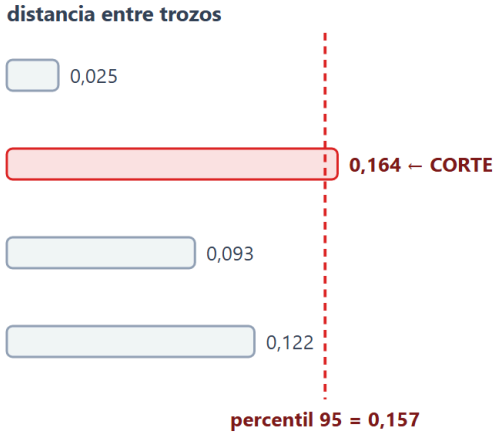
0 [Project...] [Client...] AUTH-001: nombre + descripción

1 AUTH-001: stack y horas · PSD2-002: nombre + descripción

2 PSD2-002: stack y horas · TXN-003: nombre + descripción

3 TXN-003: stack y horas · NOTIF-004: nombre + descripción

4 NOTIF-004: stack y horas



sem0 = trozos 0-1 (150 tok) · sem1 = trozos 2-4 (128 tok)

Parte el texto por «. »: cada trozo junta el final de un componente con el principio del siguiente.
Resultado: la descripción de PSD2-002 queda en un chunk y su stack y sus horas, en el otro.

En nuestro corpus: 34 chunks (17 × 2: un corte por presupuesto) · 0,000094 $ · 5,3 s de ingesta

La idea

Embebe cada frase y corta donde la siguiente se parece menos a la anterior.

Úsala para

Prosa larga que cambia de tema sin avisar.

Cuidado con

Cuesta embeddings en la ingesta. En texto estructurado, corta donde no debe.

6/8 · Propositional

Familia semántica

Propositional: un LLM reescribe el componente en frases atómicas

Ejemplo real: AUTH-001 con gpt-4o-mini → 15 proposiciones

Chunk de AUTH-001

```
[Project…] [finance…]
Component: OAuth 2.0
authentication backend
Description: OAuth 2.0
flows, JWT sessions,
multi-tenant, rate
limiting…
Stack · Complexity · Hours
```

106 tokens

LLM

0. The project focuses on a Mobile banking API.	9	8. …will implement refresh token flows.	14
1. …will use OAuth 2.0 authentication.	13	9. …will utilize JWT-based session management.	15
2. …will be compliant with PSD2 standards.	12	10. …will provide multi-tenant token isolation.	16
3. The client sector is finance.	6	11. …will include rate limiting per client.	15
4. The project is scheduled for the year 2024.	11	12. …includes Ruby on Rails, PostgreSQL…	22
5. The main technology … is Ruby on Rails.	12	13. The complexity … is high.	14
6. …is an OAuth 2.0 authentication backend.	14	14. The estimated hours … is 120 hours.	18
7. …will implement authorization code flows.	14		

Grises: hablan del proyecto, no del componente → se repiten en cada componente

15 chunks de 6 a 22 tokens: casi todos huérfanos (< 20), pero cada uno se entiende solo

En nuestro corpus: 778 chunks · 765 huérfanos · 0,00755 $ · 173 s de ingesta (una llamada por componente)

La idea

Un LLM reescribe cada componente en frases cortas que se entienden solas.

Úsala para

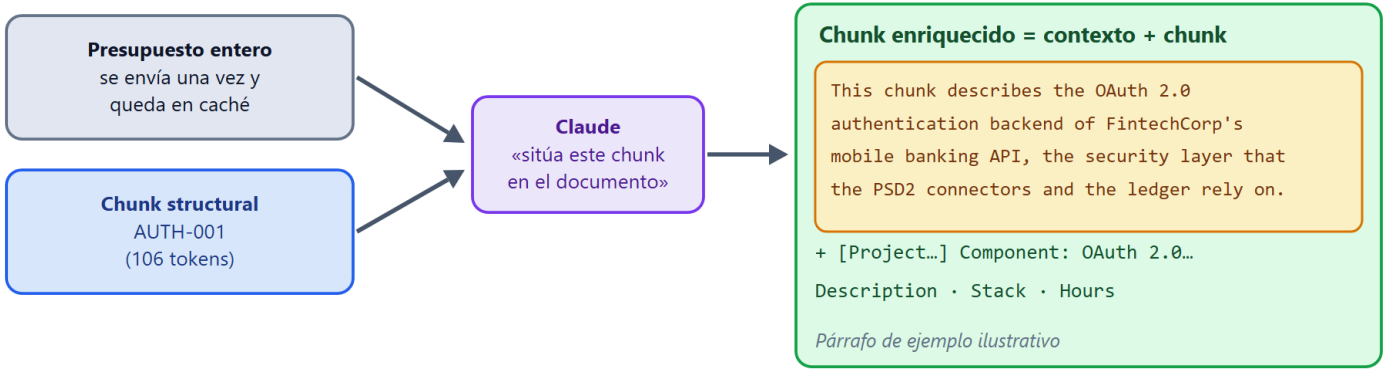
Textos densos en hechos (legal, médico) y preguntas muy concretas.

Cuidado con

Un LLM por componente. 13× más chunks, casi todos huérfanos.

Contextual Retrieval: el LLM escribe el contexto que le falta al chunk

Técnica de Anthropic · en el código, con Claude Sonnet y prompt caching



Caché: el primer chunk de cada presupuesto paga el documento (3,75 $/M); los demás lo leen a 0,30 $/M.

AUTH-001 para «OAuth authentication for fintech mobile app»: 0,568 → 0,607 (+0,04).

En nuestro corpus: 60 chunks · p50 de 132,5 tokens (78 + contexto) · 0,14 $ · 203 s de ingesta

La idea

Un LLM escribe el contexto que le falta a cada chunk y se pega delante.

Úsala para

Chunks que pierden el sentido leídos solos. Afinar un RAG que ya funciona.

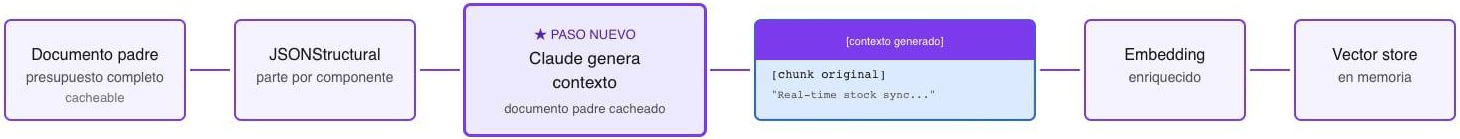
Cuidado con

La más cara: 0,14 $ y 3,4 min aquí. Nuestra cabecera ya es una versión gratis.

L1DR

Contextual Retrieval

Enriquece el chunk con un párrafo generado por Claude · prompt caching del documento padre



RESULTADO SOBRE NUESTRO CORPUS · consulta: "OAuth authentication for fintech mobile app"

JSONStructural (baseline) Top-1: BUD-2024-001::AUTH-001	0.5684	
ContextualRetrieval Mismo chunk + contexto generado por Claude	0.6070	+0.04

COSTE REAL · 60 chunks · 17 presupuestos

JSONStructural		$0.00 · 0 segundos
ContextualRetrieval		$0.140 · 202 segundos (3.4 min)

+0.04 de cosine por $0.14 · 3.4 minutos de ingesta · ¿lo aprobáis en una review?

Anthropic reporta hasta –67% de fallos de retrieval combinado con BM25 contextual + reranking

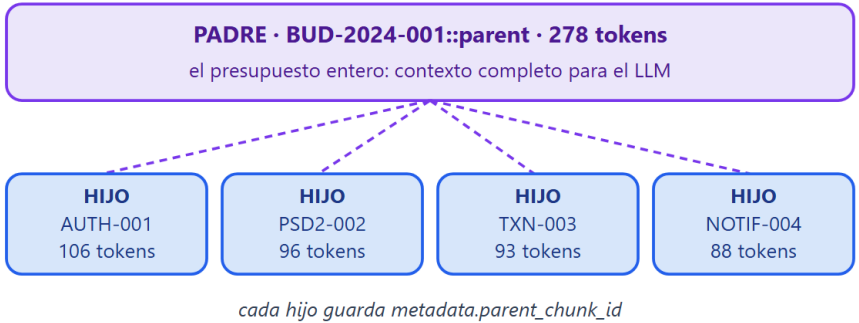
El header estático del JSONStructural ya es Contextual Retrieval barato · la versión LLM solo paga la pena en corpus donde compensa medir

8/8 · Hierarchical

Familia estructural

Hierarchical: indexa hijos y padres a la vez

Ejemplo real: BUD-2024-001 → 1 padre + 4 hijos en el mismo índice



cada hijo guarda metadata.parent_chunk_id

1 · La consulta encuentra un HIJO
pequeño y preciso → buen recall

↓

2 · Se resuelve su PADRE
parent_chunk_id → presupuesto entero

↓

3 · El LLM recibe el PADRE
con todo el contexto para responder

Consulta «GDPR compliance and audit logging» (real):

1. BUD-2024-009::CONSENT-004 (hijo) 0,418 · 2. BUD-2024-012::parent 0,307 ← un padre entero en el top-3

Para una consulta abstracta, a veces el presupuesto completo es mejor respuesta que un componente.

En nuestro corpus: 77 chunks = 17 padres + 60 hijos · 2 obesos (el padre de BUD-2024-016: 1472 tokens) · 0 $

La idea

Dos tamaños a la vez: el componente (hijo) para encontrar y el presupuesto (padre) para responder.

Úsala para

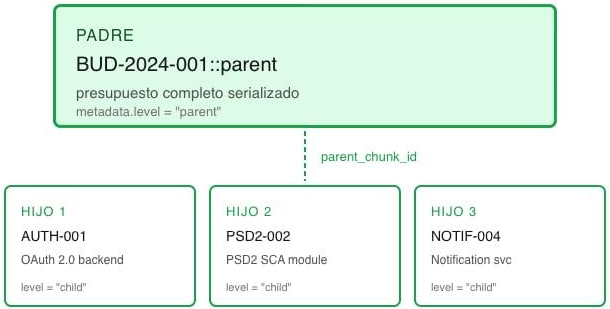
Documentos largos: manuales, contratos.

Cuidado con

Indexa más y los padres pueden ser obesos. Demasiado para FAQs.

Hierarchical · parent-child

Dos niveles indexados a la vez · el hijo da el recall, el padre da el contexto



CONSULTA: **"GDPR compliance and audit logging"**

#1 HIJO [0.4180] `BUD-2024-009::CONSENT-004`

`[Project: Healthcare data platform...] Consent and audit log...`

↓ con --resolve-parents activado, se devuelve también el padre BUD-2024-009::parent

#2 PADRE [0.3070] `BUD-2024-012::parent`

`presupuesto completo serializado`

⚠ un padre entero entra en el top-3 sin necesidad de resolver

Hijos para recall

Granularidad fina · matchea
consultas específicas con score alto.

El embedding del componente es preciso y discriminante.

Padres para contexto

Cuando el hijo matchea, se
devuelve también el padre completo.

Útil para queries abstractas: sube el padre al top-3 por sí solo.

¿Cuándo activar?

✓ Manuales largos
✓ Documentos legales
✗ FAQs cortos
✗ Tickets breves · presupuestos: borderline

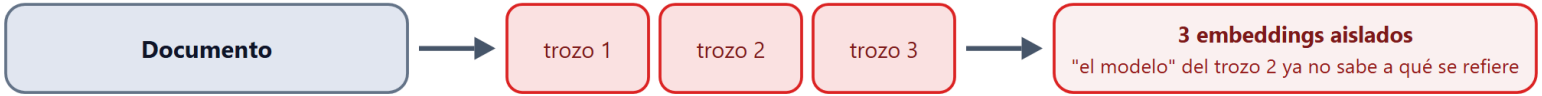
Stats: 77 chunks = 17 padres + 60 hijos · obesos: 2 (max 1472) · coste: $0

Late chunking

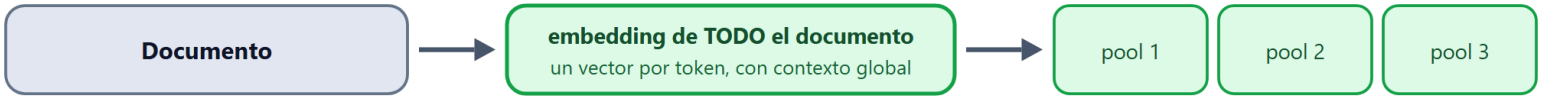
La técnica que no ejecutamos hoy: nuestro modelo no da vectores por token

Late chunking: cambiar el orden de las operaciones

Tradicional: primero se parte, luego se embebe cada trozo



Late chunking: primero se embebe el documento entero, luego se parte



cada chunk = media de SUS tokens, que ya vieron todo el documento

Qué necesita

un modelo con ventana larga (8K–32K tokens)
y que exponga embeddings por token (p. ej. jina-embeddings-v3)

En nuestro proyecto

text-embedding-3-small no expone embeddings por token:
hoy es concepto, no demo · técnica emergente (Jina, 2024)

Útil cuando el significado de cada parte depende mucho del documento completo

¿Cuándo compensa? Documentos largos, modelo con contexto grande y re-embeber barato. En nuestro caso, todavía no.

LIDR

El cuadro de mando

17 presupuestos · huérfano < 20 tokens · obeso > 800 tokens

Estrategia	Chunks	p50	Huérf.	Obesos	$	Ingesta
structural	60	78	0	1	0	0 s
fixed_size	63	79	0	0	0	0 s
recursive	23	187	1	0	0	0 s
sentence_window	78	21	34	0	0	0 s
semantic	34	112,5	0	1	0,0001	5 s
propositional	778	11	765	0	0,0076	173 s
contextual_retrieval	60	132,5	0	1	0,14	203 s
hierarchical	77	81	0	2	0	0 s

La pregunta del día

Si fuera vuestro proyecto en producción: **¿con qué estrategia salís hoy?**

Y más importante: **¿qué mediríais antes de cambiarla?**

El chunking pesa tanto como el modelo (Vectara, 2025) · más sofisticado no es mejor: recursive 1º, semantic 4º (Vecta, 2026) · **mide sobre tus datos**

L1DR

Lo que cierras hoy · lo que abre la S8

SALIMOS HOY CON

- Ocho estrategias ejecutadas sobre tu corpus
- Métricas comparativas: stats + cosine + cualitativa
- Una estrategia elegida con justificación
- Plan de qué medirías antes de cambiarla
- 60 chunks vectorizados · en memoria

EN MEMORIA · HTTP

LA SESIÓN 8 AÑADE

- PostgreSQL + pgvector
- Índices HNSW / IVFFlat para escala
- Filtrado por metadata + similitud combinada
- Búsqueda semántica real a baja latencia
- Persistencia que sobrevive al restart

PERSISTENCIA · ÍNDICES

EL ARCO COMPLETO DEL MÓDULO

SESIÓN 06
Corpus limpio
→
SESIÓN 07 ✓
Chunking + vectores
→
SESIÓN 08 ← siguiente
pgvector + retrieval
→
SESIÓN 10
Retrieval avanzado
→
SESIÓN 11
Generación y calidad

El código de hoy se queda intacto · la S8 añade una capa, no sustituye nada

✋Tiempo para el feedback con ROTI

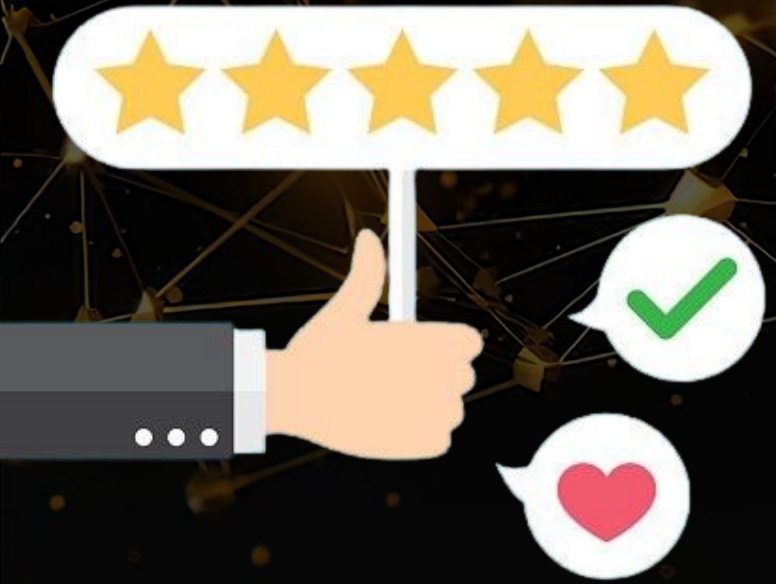
¿Cuánto valor te ha agregado esta sesión?



Absolutamente nada	Poco	Suficiente	Bastante	Más de lo que esperaba

LIDR

ROTI



¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR